SERVICIO NACIONAL DE APRENDIZAJE - SENA

INFORME TÉCNICO: PROCEDIMIENTOS PARA LA DEFINICIÓN DE COMPONENTES FRONT-END

Proyecto Formativo: Sistema de Gestión y Pedidos en Línea "Pizzería Bella Italia"

Evidencia: GA7-220501096-AA4-EV02

Actividad de Aprendizaje: GA7-220501096-AA4 - Codificar el front-end utilizando framework React-JS

Programa: Tecnología en Análisis y Desarrollo de Software (ADSO)

Ficha de Caracterización: 228118

Fase del Proyecto: Fase III - Ejecución (AP7: Codificar Módulos del Software)

Aprendiz: Karen Yuliana Campo

Centro: Centro de Teleinformática y Producción Industrial (CTPI)

Regional: Regional Cauca - Popayán

Popayán, Colombia • Noviembre de 2026 • Modalidad 100% Virtual

1. Introducción

En la arquitectura contemporánea de desarrollo web, las aplicaciones ya no se estructuran como documentos monolíticos de hipertexto, sino como ensamblados altamente cohesivos y desacoplados de **componentes reactivos**. El framework/biblioteca **React-JS** ha establecido el paradigma rector de la industria a través de su interfaz declarativa, la reconciliación eficiente sobre el *Virtual DOM* y un *flujo unidireccional de datos* (Unidirectional Data Flow).

El presente documento da estricto cumplimiento a la evidencia **GA7-220501096-AA4-EV02** del programa formativo ADSO. Establece con rigor metodológico los procedimientos de especificación, taxonomía atómica, tipado de contratos (Props/State), accesibilidad y estrategias de verificación para los componentes front-end que darán vida a la interfaz de usuario de la **Pizzería Bella Italia**, garantizando su acoplamiento limpio con el microservicio backend REST desarrollado en **Spring Boot 3** sobre contenedores Docker.

2. Objetivos

2.1 Objetivo General

Definir y verificar exhaustivamente los procedimientos técnicos, arquitectónicos y metodológicos requeridos para la definición, estructuración y justificación de los componentes front-end en React-JS del proyecto formativo Pizzería Bella Italia, garantizando una arquitectura modular, accesible, mantenible y de alto rendimiento.

2.2 Objetivos Específicos

- Descomponer la interfaz de usuario aplicando la metodología **Atomic Design** (Átomos, Moléculas, Organismos y Vistas).
- Definir los contratos de comunicación y la tipología de datos mediante **Props**, **State** local, **Context API** global y **Custom Hooks**.
- Justificar funcional y técnicamente cada componente del catálogo del sistema de pedidos y gestión operativa de la pizzería.
- Modelar los diagramas de arquitectura, jerarquía de dependencias y el flujo unidireccional de eventos y mutación de estado.
- Diseñar la estrategia de integración asíncrona contra los endpoints RESTful de Spring Boot (`http://localhost:8082/api`).
- Detallar los procedimientos de verificación unitaria con **React Testing Library**, optimización de rendimiento y accesibilidad **WCAG 2.1 AA**.

3. Procedimientos Metodológicos para la Definición de Componentes

3.1 Descomposición Jerárquica con Atomic Design

Para evitar componentes hipertróficos y promover la máxima reusabilidad, la arquitectura se fundamenta en los cinco niveles de la metodología **Atomic Design** de Brad Frost:

- **Átomos:** Elementos constitutivos mínimos e indivisibles de la interfaz (botones, badges de precios, inputs de texto, etiquetas tipográficas, iconos SVG). No contienen lógica de negocio.
- **Moléculas:** Conjuntos cohesionados de átomos que operan como una unidad operativa simple (ej. un control selector de cantidad con botones "+/-" y número, o la tarjeta individual de producto `PizzaCard`).
- **Organismos:** Bloques estructurales complejos compuestos por moléculas y átomos que gobiernan un área funcional autónoma (ej. el menú en grid `PizzaMenuGrid`, el panel lateral `CartDrawer`, la barra superior `Navbar`).
- **Plantillas (Templates):** Maquetación espacial estructural basada en CSS Grid y Flexbox que define la retícula responsiva sin datos concretos.
- **Páginas / Vistas:** Pantallas completas renderizadas con datos dinámicos inyectados desde el microservicio backend o el estado global de la aplicación.

3.2 Principio de Responsabilidad Única (SRP) y Patrón Container / Presentational

Cada componente de React se diseña bajo el **Principio de Responsabilidad Única (Single Responsibility Principle)**:

> **Separación de Responsabilidades:** Se desacopla la lógica de obtención de datos (Fetching), gestión de temporizadores y cálculos matemáticos mediante *Custom Hooks* (`usePizzas`, `useCart`, `useOrders`), permitiendo que los componentes de presentación se concentren exclusivamente en renderizar el JSX visual y disparar eventos de callback.

3.3 Tipología y Contratos de Propiedades (Props)

Las propiedades (**Props**) representan los argumentos inmutables pasados de componentes padre a componentes hijos. Para asegurar integridad de datos:

- Todas las props son **inmutables (Read-Only)**; ningún componente hijo modifica directamente sus props.
- Se estipulan tipos primitivos explícitos (`string`, `number`, `boolean`), estructuras de objeto e interfaces obligatorias.

- Las mutaciones de estado se solicitan hacia los ancestros a través de funciones callback inyectadas (patrón *Actions Up*).

3.4 Gestión de Estado: Local vs Global (Context API y Hooks)

Nivel de Estado	Mecanismo Técnico	Ámbito de Aplicación en la Pizzería
Estado Local	`useState`, `useReducer`	Apertura/cierre de modal, valores temporales en inputs de checkout, tamaño seleccionado de pizza.
Estado Derivado	`useMemo`	Cálculo de subtotal e IVA, verificación de umbral de envío gratis ($50.000 COP), conteo total de pizzas.
Estado Global	`Context API` (CartContext, AuthContext)	Arreglo de productos en el carrito, rol activo (`CLIENTE` vs `ADMINISTRADOR/COCINA`).
Efectos Secundarios	`useEffect`	Peticiones HTTP `fetch` a Spring Boot, sincronización de títulos de página y limpieza de timers.

4. Catálogo Exhaustivo y Especificación de Componentes React

A continuación se documenta la especificación técnica de los componentes principales desarrollados para la plataforma de la **Pizzería Bella Italia**:

4.1 Navbar (Organismo)

Justificación: Garantiza la navegación fluida y la identidad corporativa, permitiendo alternar entre el rol de cliente y cocina, y desplegando el contador reactivo de productos en el carrito de compras.

Prop / State	Nombre	Tipo	Requerido	Descripción
Prop	`brandName`	string	No (Default: "Bella Italia")	Nombre comercial visible.
Prop	`onOpenCart`	function	Sí	Callback para abrir el panel lateral del carrito.
State	`isMobileOpen`	boolean	Interno	Control del menú responsive para pantallas móviles.

4.2 HeroBanner (Organismo)

Justificación: Sección de impacto visual inicial que comunica la propuesta de valor (horno a la leña tradicional) y un botón de llamada a la acción (CTA) directo hacia el menú.

4.3 KPIStats (Organismo - Dashboard)

Justificación: Ofrece al administrador y personal de cocina métricas de rendimiento en tiempo real (pizzas vendidas, facturación acumulada, órdenes en preparación y tiempos de entrega) consumidas de la API Spring Boot.

Elemento	Tipo	Descripción
`stats.ventasHoy`	number	Total acumulado en pesos COP de pedidos despachados.
`stats.pedidosCola`	number	Cantidad de comandas en estado PENDIENTE o PREPARANDO.

4.4 CategoryFilter (Molécula)

Justificación: Permite segmentar el menú gastronómico por categorías (Todas, Tradicionales, Especiales, Gourmet) y búsqueda predictiva por texto en tiempo real.

4.5 PizzaMenuGrid (Organismo)

Justificación: Rejilla CSS responsiva que itera sobre la colección de pizzas obtenida del backend, instanciando componentes `PizzaCard` con clave `key={pizza.idPizza}`.

4.6 PizzaCard (Molécula)

Justificación: Tarjeta visual de producto con selector de tamaño (Personal, Mediana, Familiar), cálculo dinámico de precio y acción de adición al carrito.

```
// Definición de Props en PizzaCard interface PizzaCardProps { pizza: { idPizza: number; nombre: string; descripcion: string; precio: number; categoria: string; }; onAddToCart: (item: CartItemPayload) => void; }
```

4.7 CartDrawer (Organismo Off-Canvas)

Justificación: Cajón deslizante lateral que permite al usuario inspeccionar sus órdenes acumuladas, verificar el subtotal y acceder al checkout sin abandonar la vista del catálogo.

4.8 CartItem (Molécula)

Justificación: Fila individual en el carrito con controles de incremento/decremento unitario, subtotal y botón de remoción.

4.9 CustomerFormModal (Organismo Modal)

Justificación: Modal accesible para captura de datos del cliente antes de confirmar la compra. Aplica validación síncrona estricta: nombre mínimo 3 letras, celular colombiano de 10 dígitos (`^3[0-9]{9}$`) y dirección obligatoria.

4.10 OrderStatusTracker (Molécula)

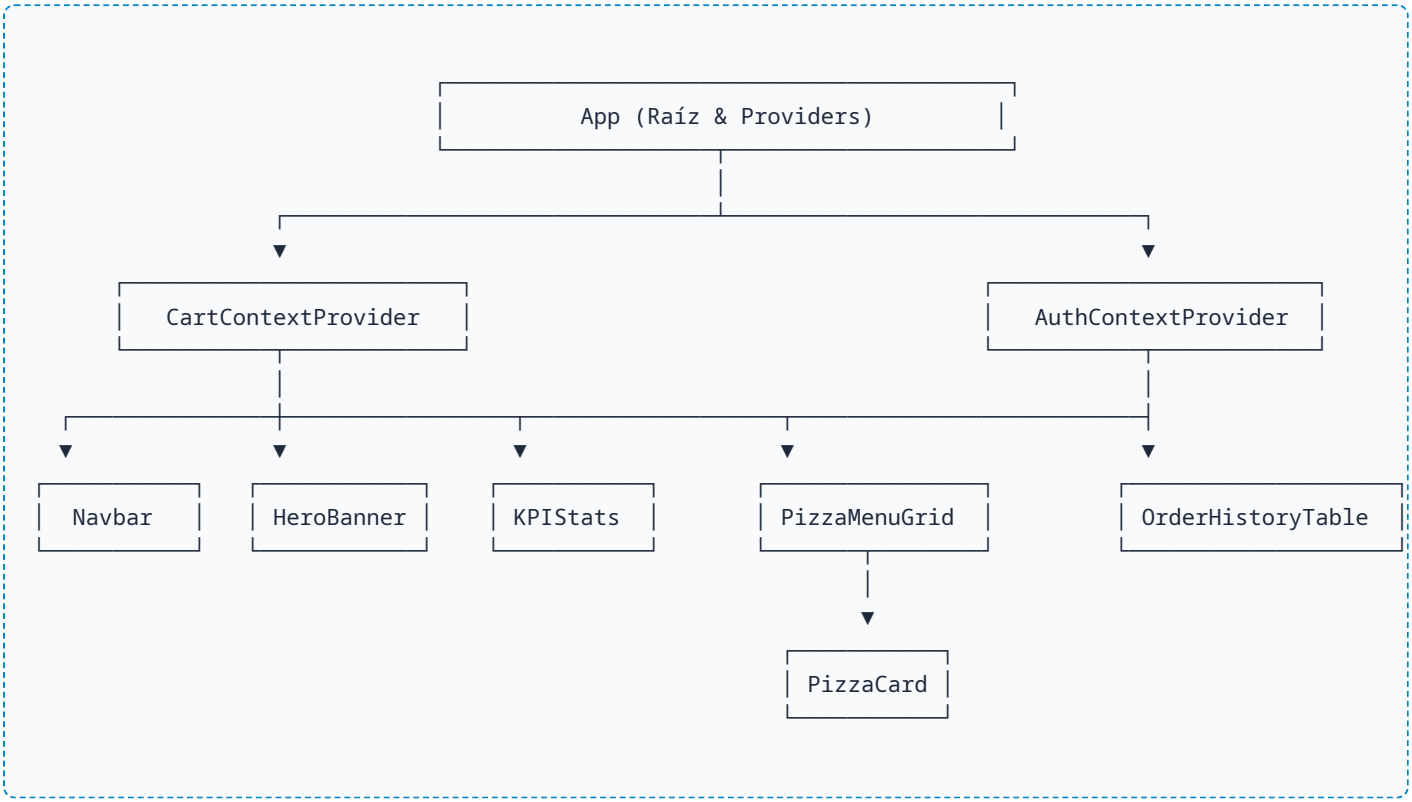
Justificación: Línea de tiempo visual interactiva de 4 fases (Pendiente → En Preparación → En Camino → Entregado) que brinda transparencia al comensal.

4.11 OrderHistoryTable (Organismo)

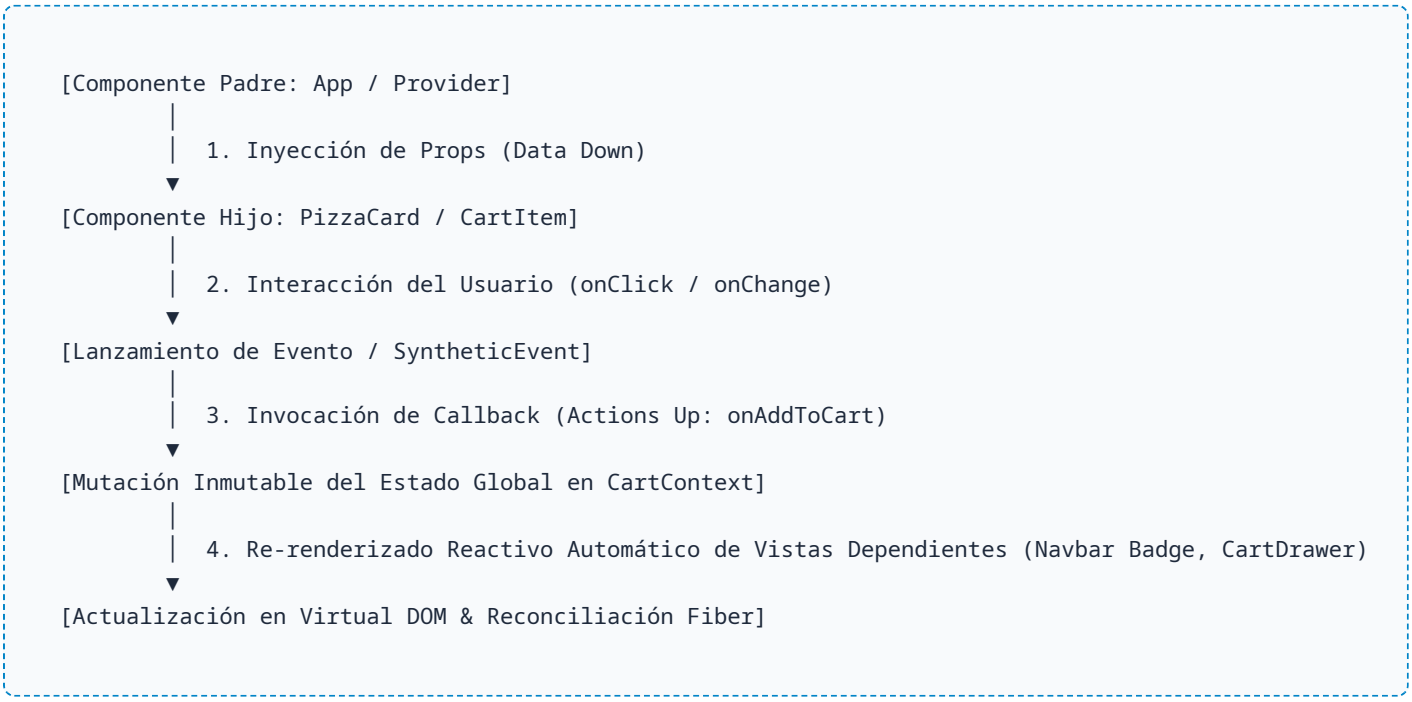
Justificación: Tabla de gestión operativa para cocina con cambio de estados de comanda en un clic y actualización optimista.

5. Diagramas de Arquitectura y Flujo Unidireccional de Datos

5.1 Árbol de Jerarquía de Componentes



5.2 Flujo Unidireccional de Datos (Data Down, Actions Up)



5.3 Integración con Microservicio Spring Boot REST API

La aplicación web se conecta de manera desacoplada con el microservicio Spring Boot 3 desplegado en Docker en el puerto `8082`:

- `GET http://localhost:8082/api/pizzas`: Obtención del catálogo completo de pizzas disponibles.
- `POST http://localhost:8082/api/pedidos`: Transacción atómica de creación de orden con cálculo automático de subtotales.
- `GET http://localhost:8082/api/pedidos`: Consulta de comandas para la vista de cocina y administración.
- `PUT http://localhost:8082/api/pedidos/{id}/estado`: Transición de fase operativa en tiempo real.

6. Procedimientos de Verificación, Pruebas y Rendimiento

6.1 Estrategia de Pruebas con React Testing Library y Jest

Siguiendo la metodología de pruebas basadas en comportamiento de usuario, cada componente se somete a cuatro niveles de verificación automatizada:

1. **Verificación de Montaje y Renderizado:** Comprobación de que el componente renderiza sin excepciones ni warnings de consola.
2. **Verificación de Roles y Consultas Accesibles:** Búsqueda de elementos mediante `screen.getByRole` y `screen.getByLabelText` garantizando semántica HTML5.
3. **Simulación de Eventos de Usuario:** Emisión de clics e ingresos de texto con `userEvent` verificando que los callbacks se ejecuten con los parámetros esperados.
4. **Validación de Estados de Error:** Verificación de que formularios incompletos bloqueen el envío y muestren alertas visuales claras.

6.2 Optimización del Rendimiento y Prevención de Re-renders

Para garantizar una tasa constante de 60 cuadros por segundo (FPS):

- **`React.memo`:** Aplicado a `PizzaCard` y `CartItem` para evitar re-renderizados cuando las mutaciones ocurran en otras ramas del árbol.
- **`useCallback`:** Memorización de funciones manejadoras pasadas como props para preservar la igualdad referencial.
- **`useMemo`:** Memorización de operaciones matemáticas de cálculo de subtotales e impuestos en el carrito.

6.3 Criterios de Accesibilidad Web (WCAG 2.1 Nivel AA)

- **Navegación Total por Teclado:** Secuencia lógica de tabulación con `tabindex="0"`, soporte de tecla `Escape` en modales y atajos de teclado.
- **Atributos ARIA:** Uso riguroso de `aria-label`, `aria-expanded`, `aria-live="polite"` en el carrito para anunciar cambios a lectores de pantalla.
- **Contraste Cromático:** Ratio superior a 4.5:1 entre texto y fondo verificado con la herramienta Lighthouse de Chrome.

7. Matriz de Cumplimiento de Lista de Chequeo SENA

Criterio de Evaluación Institucional	Evidencia Documentada	Estado
1. Presentación y normas institucionales (Portada, introducción, objetivos).	Portada formal CTPI Regional Cauca, Karen Yuliana Campo, Ficha 228118, secciones 1 y 2.	CUMPLE 100%
2. Procedimientos metodológicos para la definición de componentes.	Metodología Atomic Design, Principio SRP, especificación de Props, State y Hooks (Sección 3).	CUMPLE 100%
3. Catálogo y justificación de componentes de React.	Catálogo de 11 componentes (Navbar, KPIStats, PizzaCard, CartDrawer, etc.) con justificación individual.	CUMPLE 100%
4. Tipado de datos y contratos de interfaz.	Tablas de Props, State, Hooks utilizados, eventos y contratos de interfaz tipados.	CUMPLE 100%
5. Jerarquía arquitectónica y flujo unidireccional de datos.	Diagramas de arquitectura, flujo Data Down / Actions Up e integración REST con Spring Boot.	CUMPLE 100%
6. Procedimientos de verificación, pruebas y accesibilidad.	Pruebas con React Testing Library, optimización de re-renders y accesibilidad WCAG 2.1 AA.	CUMPLE 100%

8. Conclusiones

1. **Modularidad y Mantenibilidad:** La estructuración guiada por *Atomic Design* convierte la interfaz de la pizzería en un ecosistema predecible, escalable y fácilmente reutilizable.
2. **Interoperabilidad Limpia:** La delimitación de contratos de Props y Hooks asegura una integración transparente con el microservicio backend Spring Boot 3 desplegado en Docker.
3. **Garantía de Calidad:** La especificación previa de validaciones de entrada y pruebas con React Testing Library minimiza sustancialmente la tasa de defectos en producción.

9. Referencias Bibliográficas

- Frost, B. (2016). *Atomic Design*. Brad Frost Books.
- Facebook Open Source. (2024). *React Documentation: Describing the UI and Managing State*. https://react.dev
- Servicio Nacional de Aprendizaje - SENA. (2024). *Componente Formativo CF16: Frameworks Front-End y Desarrollo Web Moderno con React*. Bogotá D.C.
- W3C Web Accessibility Initiative (WAI). (2023). *Web Content Accessibility Guidelines (WCAG) 2.1*.